Polynomial System Solving via Factor Branching and Recursive Splitting

Ilay Menahem[a], Michael Shalyt[a] and Ido Kaminer[a,b]

[a]Andrew and Erna Viterbi Faculty of Electrical and Computer Engineering, Technion – Israel Institute of Technology, Haifa, 3200003, Israel
[b]Department of Materials Science and Engineering, Technion – Israel Institute of Technology, Haifa, 3200003, Israel

ARTICLE INFO

Keywords:
Polynomial systems
Gröbner bases
Factorization
Minimal hitting sets
Recursive splitting

ABSTRACT

Polynomial systems are a fundamental language for describing algebraic and geometric problems in mathematics and its applications. Beyond finding individual solutions, one often seeks global properties of the solution space, such as its dimension and irreducible components, or structural properties and decompositions of the underlying polynomial ideal. For large polynomial systems with many variables and high degrees, determining these global properties can become intractable. Leading approaches include variants of Buchberger's algorithm, and resultants. However, even the most modern approaches still often fail on practical problems or slight perturbations of standard benchmarks. Here we introduce a framework that divides a polynomial system into simpler branches by interreducing and factoring its equations. The framework only assumes that the solver used on each branch describes satisfiable (SAT) branches and certifies unsatisfiable (UNSAT) ones. When individual branch computations exceed time limits, the framework refines stalled branches through recursive variable splitting, illustrated here by zeroing ($x = 0$ versus $x \neq 0$) to isolate simpler sub-varieties. In our experiments, we use Gröbner-basis solvers and apply the method to a system arising from research in quantum field theory for which no standard method in our comparison succeeds. Together, these ideas offer a path toward the exact analysis of previously inaccessible polynomial systems.

1. Introduction

Systems of multivariate polynomial equations appear whenever a symbolic model contains nonlinear algebraic constraints. Applications include robotics, coding theory, optimization, statistics, and physics (Sturmfels, 2002). In many of these problems, an isolated solution is not enough. One needs the dimension of the solution set, its irreducible components, or the exact relations that hold on every solution. These are questions about the variety and its defining ideal, rather than about a single point (Cox et al., 2015; Kreuzer and Robbiano, 2000).

Elimination by resultants provided an early systematic way to attack polynomial systems, but the introduction of Gröbner bases gave a common algorithmic framework for elimination, ideal membership, dimension, and related tasks (Cox et al., 2015; Kreuzer and Robbiano, 2000). Buchberger developed this framework in his 1965 thesis (Buchberger, 2006). Later algorithms changed how the expensive reductions are organized. F4 replaces individual polynomial reductions with sparse linear algebra, while F5 uses signatures to avoid many reductions to zero (Faugère, 1999, 2002). Primary-decomposition algorithms use Gröbner bases and factorization to separate components (Gianni et al., 1988). Factorizing Gröbner-basis methods, split on individual polynomial factorizations and preserve the solution set; earlier methods also interreduce and prune redundant branches (Decker et al., 2025; Gräbe, 1995). Numerical algebraic geometry takes a different route: homotopy continuation computes isolated roots numerically, while witness sets represent positive-dimensional components (Sommese et al., 2001; Sommese and Wampler, 2005).

Recent work has also explored machine-learning methods for polynomial systems. Peifer et al. trained a reinforcement-learning policy for pair selection in Buchberger's algorithm (Peifer et al., 2020). Kera et al. trained transformers to compute Gröbner bases directly for a restricted class of zero-dimensional radical ideals (Kera et al., 2024). Yet these works are limited to small and specific polynomial ideals.

Despite this progress, large systems such as a system that came up in our research of quantum gravity can remain inaccessible.

Here, we introduce a solver-agnostic framework that breaks down a polynomial system into smaller branches by factoring and interreducing its equations. The framework enumerates the minimal hitting sets of the resulting factor sets; each hitting set contains at least one factor from every equation and therefore defines one branch of the original solution set. An exact branch solver then either describes that branch, including any positive-dimensional families, or

certifies that it is UNSAT. Since the worst-case complexity of computing a Gröbner basis is doubly exponential in the number of variables (Mayr and Meyer, 1982), breaking down a large system into smaller branches can be substantially faster than solving the original system directly.

If a factor branch remains too expensive for the chosen solver, the framework can refine it by splitting a selected variable into the cases $x = 0$ and $x \neq 0$. This heuristic split can expose additional algebraic structure for the branch solver without losing any solutions. While this is one of many possible splitting criteria, it proves particularly effective on the quantum field theory problem that we investigate.

We prove that refinement and variable zeroing terminate and preserve the full solution set. We compare the resulting implementation with direct Gröbner-basis computation and symbolic decomposition on generated benchmarks. We also apply the framework to a system from quantum field theory: with the `facstd` backend, the framework completes the computation, whereas standard standalone solvers timed out.

2. Preliminaries and notation

2.1. Polynomial systems, ideals, and varieties

Let K be a field, let $\overline{K}$ be its algebraic closure, and write $R = K[x_1, \dots, x_n]$. The input is a finite system $P = \{p_1, \dots, p_m\} \subseteq R$ of nonzero polynomials. Unless stated otherwise, a solution is a point in $\overline{K}^n$. For a finite set $Q \subseteq R$, its solution set is

$$V(Q) = \{a \in \overline{K}^n : q(a) = 0 \text{ for every } q \in Q\}.$$

We call Q satisfiable (SAT) if $V(Q) \neq \emptyset$ and unsatisfiable (UNSAT) if $V(Q) = \emptyset$.

The ideal generated by Q, denoted $\langle Q \rangle$, consists of all sums $\sum_{q \in Q} h_q q$ with $h_q \in R$.

For $X \subseteq \overline{K}^n$, its vanishing ideal $I(X)$ contains every polynomial in R that vanishes on all of X. The radical of an ideal J is $\sqrt{J} = \{g \in R : g^r \in J \text{ for some } r \geq 1\}$. Hilbert's Nullstellensatz gives

$$I(V(Q)) = \sqrt{\langle Q \rangle}.$$

Thus $V(Q) = \emptyset$ exactly when $\langle Q \rangle = \langle 1 \rangle$. These facts connect solution sets with computations in polynomial ideals (Cox et al., 2015; Kreuzer and Robbiano, 2000).

2.2. Monomial orders and Gröbner bases

Let $\mathbb{M}_n = \{x^\alpha : \alpha \in \mathbb{N}^n\}$ be the monomials of R. A monomial order $\prec$ is a total order on $\mathbb{M}_n$ such that every nonempty subset has a least element and

$$u \prec v \implies uw \prec vw \qquad (u, v, w \in \mathbb{M}_n).$$

For a nonzero polynomial p, the leading monomial $\mathrm{LM}(p)$ is the greatest monomial of p under $\prec$, $\mathrm{LC}(p)$ is its coefficient, and $\mathrm{LT}(p)$ is its leading term.

For an ideal J, define its leading-monomial ideal by

$$\mathrm{LM}(J) = \langle \mathrm{LM}(f) : f \in J \setminus \{0\} \rangle.$$

A finite set $G \subseteq J$ is a Gröbner basis of J if

$$\mathrm{LM}(J) = \langle \mathrm{LM}(g) : g \in G \rangle.$$

A finite set Q is *interreduced* if every element is monic and no term of one element is divisible by the leading monomial of another. Algorithm 1 interreduces a finite set of nonzero polynomials.

Algorithm 1 Interreduction

Require: A finite set $Q \subseteq R \setminus \{0\}$ and a monomial order $\prec$
Ensure: An interreduced set that generates $\langle Q \rangle$
while there exist distinct $p, q \in Q$ and a term cu of p with $\mathrm{LM}(q) \mid u$ **do**
  $r \leftarrow p - \dfrac{cu}{\mathrm{LT}(q)} q$
  $Q \leftarrow Q \setminus \{p\}$
  if $r \neq 0$ **then**
    $Q \leftarrow Q \cup \{r\}$
  end if
end while
$Q \leftarrow \{p/\mathrm{LC}(p) : p \in Q\}$
return Q

Each loop iteration cancels one monomial u and introduces only monomials strictly smaller than u. The multiset extension of the monomial order is well-founded, so the loop terminates. Interreduction preserves $\langle Q \rangle$ and $V(Q)$ (Cox et al., 2015, Ch. 2, §7).

Every ideal has a unique reduced Gröbner basis for a fixed monomial order. The reduced basis is $\{1\}$ if and only if $J = \langle 1 \rangle$. Therefore, a Gröbner basis computation can describe a branch and detect when it is UNSAT (Cox et al., 2015; Kreuzer and Robbiano, 2000).

2.3. Factor sets and hitting sets

Fix a factorization of each input polynomial over K:

$$p_i = a_i \prod_{f \in C_i} f^{e_{i,f}},$$

where $a_i \in K \setminus \{0\}$, C_i contains the distinct monic, nonconstant irreducible factors of p_i, and $e_{i,f} \geq 1$. Multiplicity does not affect the zero set. A point satisfies $p_i = 0$ exactly when at least one factor in C_i vanishes at that point.

A hitting set for $C_1, \ldots, C_m$ is a set $H \subseteq \bigcup_i C_i$ such that $H \cap C_i \neq \emptyset$ for every i. It is minimal if no proper subset is also a hitting set. We write $\mathcal{H}(P)$ for the family of minimal hitting sets obtained from P. Each $H \in \mathcal{H}(P)$ defines a branch in which all factors in H vanish (Berge, 1989).

2.4. Encoding nonzero variables

Let $N \subseteq \{x_1, \ldots, x_n\}$ be the variables required to be nonzero. Here are two ways to encode these constraints as polynomial equations.

One encoding introduces a separate auxiliary variable s_x for each $x \in N$ and works in the extended ring $R_{\mathrm{sep}} = R[s_x : x \in N]$. For each $x \in N$, define the polynomial

$$\sigma_x = 1 - s_x x, \tag{1}$$

and impose the equation $\sigma_x = 0$. This is the Rabinowitsch trick (Rabinowitsch, 1930).

The second encoding uses one auxiliary variable s, works in $R_{\mathrm{prod}} = R[s]$, and combines all nonzero constraints into the polynomial

$$\sigma_N = 1 - s \prod_{x \in N} x, \tag{2}$$

with the equation $\sigma_N = 0$.

Let $Q \subseteq R$ and let π denote the appropriate coordinate projection onto the original $x_1, \ldots, x_n$ coordinates. In the first encoding,

$$\pi\left(V\left(Q \cup \{\sigma_x : x \in N\}\right)\right) = \{a \in V(Q) : a_x \neq 0 \text{ for every } x \in N\}.$$

and the product encoding satisfies

$$\pi\left(V\left(Q \cup \{\sigma_N\}\right)\right) = \{a \in V(Q) : a_x \neq 0 \text{ for every } x \in N\}.$$

3. Factor branching framework

3.1. Framework description

Polynomial-ideal computations can be intractable even for small systems (Mayr and Meyer, 1982). Thus, decomposing a system into smaller and simpler branches may make the individual solver calls easier.

We break a system into smaller branches by factoring each generator and enumerating the minimal hitting sets of the factor sets. Each hitting set generates a new branch, which we interreduce and factor again, until no branch changes.

Algorithm 2 first factors P and then applies these two operations to every branch until it reaches a fixed point. We call a branch *terminal* if one round leaves it unchanged. Finally, it drops the terminal branches whose solutions lie among those of another (Subsection 3.2).

We note $\mathcal{H}(P)$ as the minimal hitting sets of the factor sets.

Algorithm 2 Refinement

Require: A finite polynomial system P
Ensure: The terminal branches of P
$W \leftarrow \mathcal{H}(P)$
repeat
  $W_{\text{prev}} \leftarrow W, \quad W \leftarrow \emptyset$
  for all $Q \in W_{\text{prev}}$ **do**
    $Q \leftarrow \textsc{Interreduce}(Q)$
    $W \leftarrow W \cup \mathcal{H}(Q)$
  end for
until $W = W_{\text{prev}}$
$W \leftarrow \textsc{Prune}(W)$ ▷ Subsection 3.2
return W

Let $\mathcal{L}(P)$ denote the terminal branches returned by Algorithm 2 on P. For each $H \in \mathcal{L}(P)$, the branch solver must either describe $V(H)$ exactly or certify that $V(H) = \emptyset$.

Algorithm 3 Factor branching

Require: A set of polynomials $P = \{p_1, \ldots, p_m\}$ and an exact branch solver $\mathcal{B}$.
$S \leftarrow \emptyset$ ▷ Solution-set representations
for all $H \in \textsc{Refine}(P)$ **do**
  Use $\mathcal{B}$ to solve H
  if $\mathcal{B}$ returns a representation of $V(H)$ **then**
    Add the returned representation to S
  end if
end for
return S

The order of the terminal branches affects running time but not correctness. Optionally, we can sort the branches by their estimated cost to solve. We use the number of generators in a branch as a proxy for the cost, processing smaller branches first.

3.2. Pruning contained branches

Hitting sets are minimal only among the children of one branch, so the loop of Algorithm 2 can end with distinct branches H and H' such that $V(H) \subseteq V(H')$. Solving H then adds no solutions. If every generator of H' reduces to zero modulo H, then $H' \subseteq \langle H \rangle$ (Cox et al., 2015, Ch. 2, §3), which certifies $V(H) \subseteq V(H')$.

$\textsc{Prune}(W)$ returns the branches it can't certify there solution sets to be contained in each other. Factorizing Gröbner-basis methods also discard redundant branches (Gräbe, 1995).

3.3. Theoretical analysis

Let S be a collection of solution-set representations, and $\mathrm{Sol}(S)$ be the union of the represented sets.

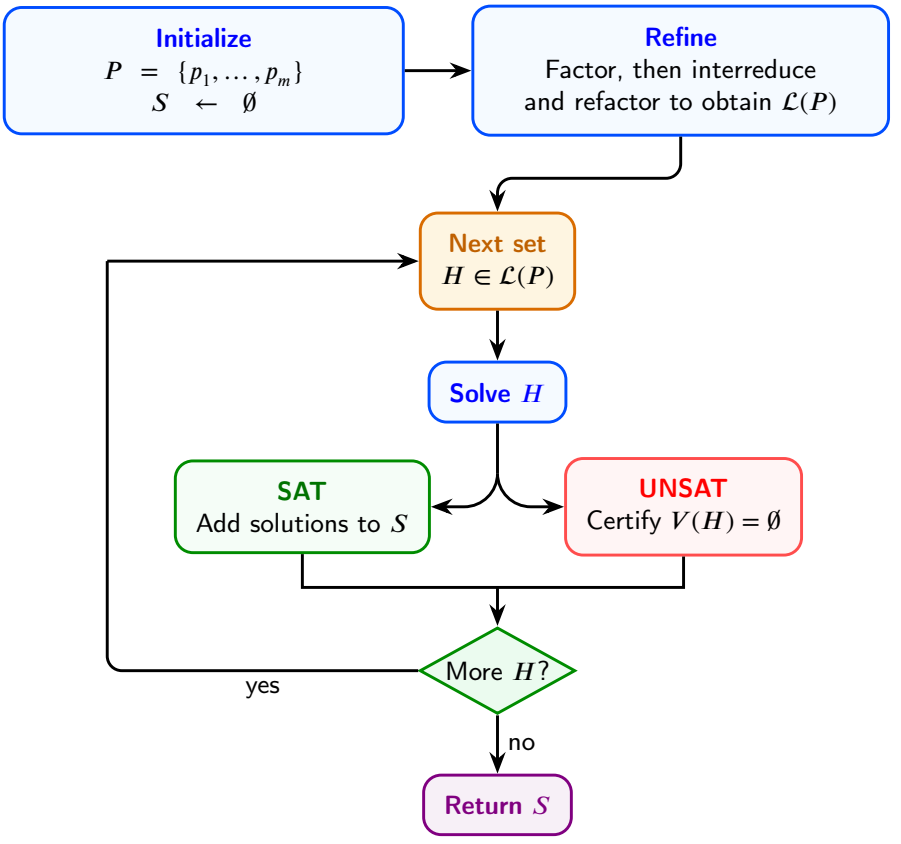


Figure 1: Factor branching procedure of Algorithm 3. Refinement produces terminal branches, which the exact solver either represents or certifies as UNSAT.

Theorem 1 (One-round branch decomposition).

$$V(Q) = \bigcup_{H \in \mathcal{H}(Q)} V(H).$$

PROOF. Write $Q = \{q_1, \ldots, q_m\}$ and let C_i be the monic nonconstant irreducible factors of q_i. Since $V(fg) = V(f) \cup V(g)$ (Cox et al., 2015, Ch. 4, §3), $V(Q) = \bigcap_i \bigcup_{f \in C_i} V(f)$. Distributing the intersection over the unions and keeping the minimal terms gives the union over the minimal transversals of $C_1, \ldots, C_m$ (Eiter and Gottlob, 1995); this is the splitting step of factorizing Gröbner-basis methods (Gräbe, 1995).

If some q_i is a nonzero constant, then $C_i = \emptyset$, and both sides are empty: $V(Q) = \emptyset$, and $\mathcal{H}(Q) = \emptyset$ because no set meets C_i. □

For a branch Q, write $\widetilde{Q}$ for the system that one round of Algorithm 2 factors: $Q \setminus \{0\}$ made monic and interreduced. Thus Q is terminal if and only if $\mathcal{H}(\widetilde{Q}) = \{Q\}$. To prove that Algorithm 2 terminates, we measure the progress of a branch Q by the leading-monomial ideal of its normalization,

$$\mu(Q) = \langle \mathrm{LM}(q) : q \in \widetilde{Q} \rangle.$$

Lemma 1 (Progress of one refinement step). *Let $H \in \mathcal{H}(\widetilde{Q})$. Then H is terminal, or $\mu(Q) \subsetneq \mu(H)$.*

PROOF. Every element of H is a monic irreducible factor of some element of $\widetilde{Q}$. Hence H contains neither 0 nor a constant, and each factor set of H is a singleton $\{h\}$. Its only minimal hitting set is H itself.

If $H = \widetilde{Q}$, then H is interreduced, so $\widetilde{H} = H$ and H is terminal. Assume $H \neq \widetilde{Q}$; we show $\mu(Q) \subsetneq \mu(H)$ in two steps.

First, $\mu(Q) \subsetneq \langle \mathrm{LM}(h) : h \in H \rangle$. Since H meets the factor set of every $q \in \widetilde{Q}$, some $h \in H$ divides q, and then $\mathrm{LM}(h) \mid \mathrm{LM}(q)$. This proves the inclusion. Suppose, for contradiction, that equality holds. Take $h \in H$ and $q \in \widetilde{Q}$ with $h \mid q$. Equality gives some $q' \in \widetilde{Q}$ with $\mathrm{LM}(q') \mid \mathrm{LM}(h) \mid \mathrm{LM}(q)$. Since $\widetilde{Q}$ is interreduced, no term of q is divisible by the leading monomial of another element, so $q' = q$. Hence $\mathrm{LM}(h) = \mathrm{LM}(q)$, the cofactor q/h is a constant, and $h = q$ because both are monic. Thus $H \subseteq \widetilde{Q}$. Conversely, every $q \in \widetilde{Q}$ is divisible by some $h \in H$, and $h \in \widetilde{Q}$ by the previous argument; interreducedness again forces $h = q$, so $\widetilde{Q} \subseteq H$. This contradicts $H \neq \widetilde{Q}$.

Second, interreduction does not shrink the ideal generated by the leading monomials. Consider one iteration of Algorithm 1, which cancels a term cu of p using q. If $u = \mathrm{LM}(p)$, then $\mathrm{LM}(p)$ remains in the ideal because q remains in the set and $\mathrm{LM}(q) \mid u$. Otherwise $u \prec \mathrm{LM}(p)$, the subtracted multiple of q has only monomials $\preceq u$, and the new element r keeps the leading monomial of p. The final normalization does not change leading monomials. Therefore $\langle \mathrm{LM}(h) : h \in H \rangle \subseteq \mu(H)$, and the two steps give $\mu(Q) \subsetneq \mu(H)$. □

Theorem 2 (Termination of refinement). *Algorithm 2 terminates on every finite polynomial system.*

Proof. By Lemma 1, μ strictly increases along every chain of nonterminal branches. The ascending-chain condition bounds such chains, and each branch has finitely many children, so the iteration reaches its fixed point after finitely many rounds. We make this precise with a tree.

Let the refinement tree of P have root P with children $\mathcal{H}(P)$. The children of any other nonterminal node Q are the elements of $\mathcal{H}(\widetilde{Q})$, and other terminal nodes have no children. Consider a path $Q_0, Q_1, \ldots$ from the root. For $i \geq 1$, every node Q_{i+1} that is not terminal satisfies $\mu(Q_i) \subsetneq \mu(Q_{i+1})$ by Lemma 1, and only the last nonroot node of a path can be terminal. The monomial ideals $\mu(Q_1) \subsetneq \mu(Q_2) \subsetneq \cdots$ therefore form a strictly ascending chain, which is finite by Dickson's lemma (Cox et al., 2015, Ch. 2, §4). Thus every path is finite. Every node has finitely many children, because its factor sets are finite. By König's lemma (Kőnig, 1927), the tree is finite; let d be its depth.

Let W_k be the value of W after k rounds of the loop in Algorithm 2. Induction on k shows that W_k is the set of nonroot terminal nodes of depth at most k together with all nodes of depth exactly $k+1$: a terminal node Q satisfies $\mathcal{H}(\widetilde{Q}) = \{Q\}$ and stays, and a nonterminal node of depth $k+1$ is replaced by its children. Hence W_k equals the set $\mathcal{T}$ of all nonroot terminal nodes for every $k \geq d$, and the loop stops after at most $d+1$ rounds. If it stops earlier, with $W_{k+1} = W_k$, then $W_j = W_k$ for all $j \geq k$ because each round depends only on W. In particular $W_k = W_d = \mathcal{T}$. In both cases, the loop ends with the finite set $\mathcal{T}$ of terminal branches, and Algorithm 2 returns a subset of it. □

Lemma 2 (Refinement preserves solutions). *Every round of Algorithm 2 preserves* $\bigcup_{Q \in W} V(Q)$. *Consequently,*

$$V(P) = \bigcup_{H \in \mathcal{L}(P)} V(H).$$

Proof. Removing the zero polynomial, interreduction, and scaling by nonzero constants preserve the zero set, so $V(Q) = V(\widetilde{Q})$. Theorem 1 applied to $\widetilde{Q}$ gives

$$V(Q) = \bigcup_{H \in \mathcal{H}(\widetilde{Q})} V(H)$$

for every $Q \in W$. Taking the union over $Q \in W$ shows that one round preserves $\bigcup_{Q \in W} V(Q)$. Before the first round, $W = \mathcal{H}(P)$, and $\bigcup_{H \in \mathcal{H}(P)} V(H) = V(P)$ by Theorem 1. By Theorem 2, the loop ends after finitely many rounds with $W = \mathcal{L}(P)$, which proves the equality. □

We call a collection S of solution-set representations *sound* for P if every represented point is a solution, $\mathrm{Sol}(S) \subseteq V(P)$, and *complete* for P if every solution is represented, $V(P) \subseteq \mathrm{Sol}(S)$. The following theorem allows the use of Algorithm 3 without loss of correctness.

Theorem 3. *If $\mathcal{B}$ returns on every branch in $\mathcal{L}(P)$, then the output of Algorithm 3 is sound and complete for P.*

Proof. By Theorem 2, $\mathcal{L}(P)$ is finite, and by Lemma 2, $V(P) = \bigcup_{H \in \mathcal{L}(P)} V(H)$. For each $H \in \mathcal{L}(P)$, the solver returns either an exact representation of $V(H)$, which the algorithm adds to S, or a certificate that $V(H) = \emptyset$. Hence

$$\mathrm{Sol}(S) = \bigcup_{H \in \mathcal{L}(P)} V(H) = V(P).$$

□

Corollary 1. *If $\mathcal{B}$ returns on every input, then Algorithm 3 terminates.*

PROOF. By Theorem 2, REFINE(P) returns the finite set $\mathcal{L}(P)$, so the algorithm makes finitely many solver calls, each of which returns. □

4. Recursive splitting

4.1. Factor branching with variable zeroing

Suppose the solver does not terminate within a reasonable amount of time on a branch produced by Algorithm 3.

Then we can further simplify the system by recursive splitting. In each child branch, we assume a side condition and run Algorithm 3 on the simplified system. This process continues until every branch is solved or a global time budget is exhausted.

For completeness, the side conditions must cover all possibilities. They should also simplify the system for the solver input and work well with Algorithm 2.

In this work, we use *variable zeroing*: recursive splitting with the side conditions $x = 0$ and $x \neq 0$ for a selected variable x. Setting $x = 0$ deletes every monomial containing x, which can make generators factor and so create new hitting sets. Requiring $x \neq 0$ removes the components of the branch that lie in $\{x = 0\}$.

Algorithm 4 Factor branching with variable zeroing

Require: A polynomial system P, a branch solver $\mathcal{B}$, a per-call timeout τ, and a global budget β

$S \leftarrow \emptyset$; $U \leftarrow \emptyset$ ▷ Solved and unresolved branches
$W \leftarrow \{(P, \emptyset, \emptyset)\}$ ▷ Pending branches (Q, Z, N)
while $W \neq \emptyset$ **do**
  Remove an element (Q, Z, N) from W
  for all $H \in$ REFINE(Q) **do**
    Run $\mathcal{B}$ on H for at most τ, unless β is exhausted
    $V \leftarrow \mathrm{vars}(H) \setminus (Z \cup N)$ ▷ Useful variables for H
    if $\mathcal{B}$ returns a representation of $V(H)$ **then**
      Add the representation, with (Z, N), to S
    else if β is exhausted or $V = \emptyset$ **then**
      $U \leftarrow U \cup \{(H, Z, N)\}$
    else
      $x \leftarrow$ CHOOSEVARIABLE(H, V)
      $W \leftarrow W \cup \{(H|_{x=0}, Z \cup \{x\}, N)\}$ ▷ $x = 0$
      $W \leftarrow W \cup \{(\text{UPDATESATURATION}(H, N \cup \{x\}), Z, N \cup \{x\})\}$ ▷ $x \neq 0$
    end if
  end for
end while
return (S, U)

Each pending branch (Q, Z, N) records the variables Z set to zero and the variables N declared nonzero on its path. Subsection 2.4 describes two ways to encode the nonzero conditions, and some solvers have their own internal encoding.

Lemma 3. *Refinement keeps exactly one generator containing s_x, of the form $c(1 - s_x h_x)$ with $h_x \equiv x$ modulo the auxiliary-free generators H'. Hence $\pi(V(H)) = \{a \in V(H') : a_x \neq 0,\ x \in N\}$, and* UpdateSaturation *loses no constraint.*

PROOF. The leading monomial of $1 - s_x h_x$ contains s_x, so it reduces no other generator, is reduced only by H' (changing h_x by an element of $\langle H' \rangle$), and, being linear in s_x, does not factor. □

Solutions of the nonzero branch are projected onto the original variables, so the auxiliary variables do not appear in the output. The zero and nonzero decisions remain part of the solution-set representation.

For comparison, we repeat the experiment twice: once with the fixed side conditions $x = 1$ and $x \neq 1$, and once with each split independently drawing an integer a uniformly from $\{-10, \ldots, 10\}$ and using the side conditions $x = a$

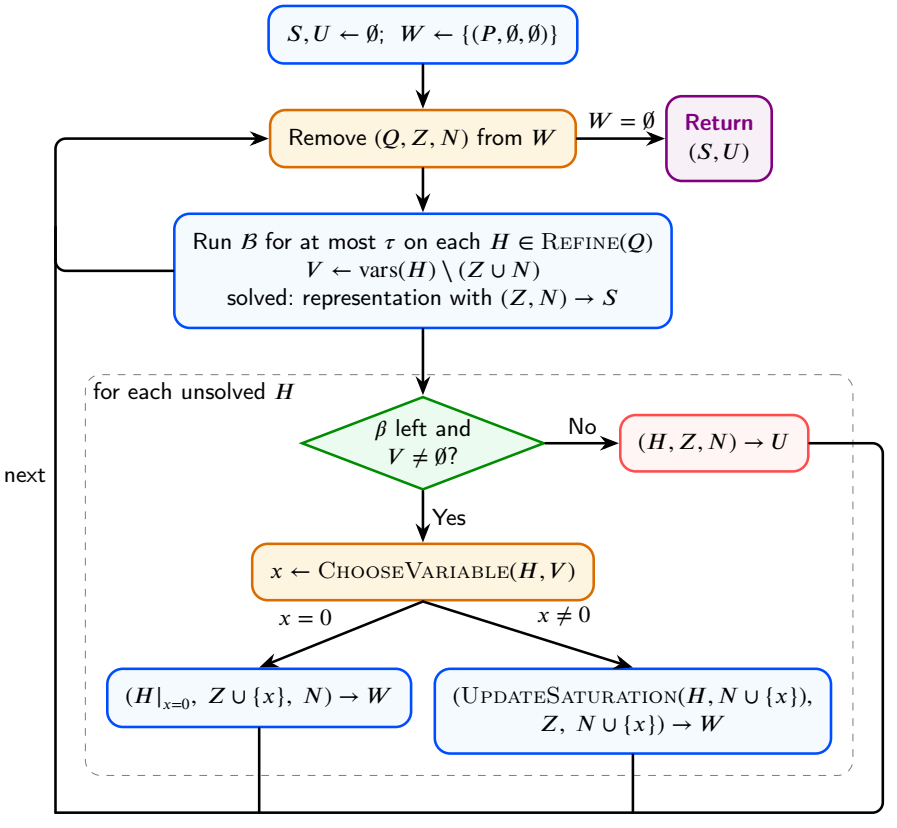


Figure 2: Flowchart of Algorithm 4.

and $x \neq a$. Both runs have a ten-minute wall-clock limit. Neither alternative finishes within ten minutes, and in both cases the search has branches that remain unresolved even after exhausting every variable.

These comparisons demonstrate that zero/nonzero splits are substantially more effective than arbitrary affine splittings on these structured systems.

4.2. Choice of variable for splitting

The choice of variable for splitting affects the efficiency of the algorithm. If a variable barely occurs in the current branch, its child branches may repeat the same work. A variable that occurs in many generators is a more promising choice. Setting it to zero removes all terms containing it at once. In the nonzero branch, UpdateSaturation adds the equation $\sigma_x = 0$.

For each $x \in X = \text{vars}(Q) \setminus (Z \cup N)$, the variables not yet split on, define

$$g_Q(x) = \#\{q \in Q : x \text{ occurs in } q\},$$
$$t_Q(x) = \#\{(q, m) : q \in Q,\ m \text{ is a term of } q,\ \deg_x(m) > 0\},$$
$$e_Q(x) = \sum_{q \in Q} \sum_{\substack{m \text{ a term of } q, \\ \deg_x(m) > 0}} \deg_x(m).$$

Its score is the triple

$$h_Q(x) = \big(g_Q(x), t_Q(x), e_Q(x)\big),$$

and scores are compared lexicographically. ChooseVariable(Q, X) returns a variable $x \in X$ with the largest score $h_Q(x)$.

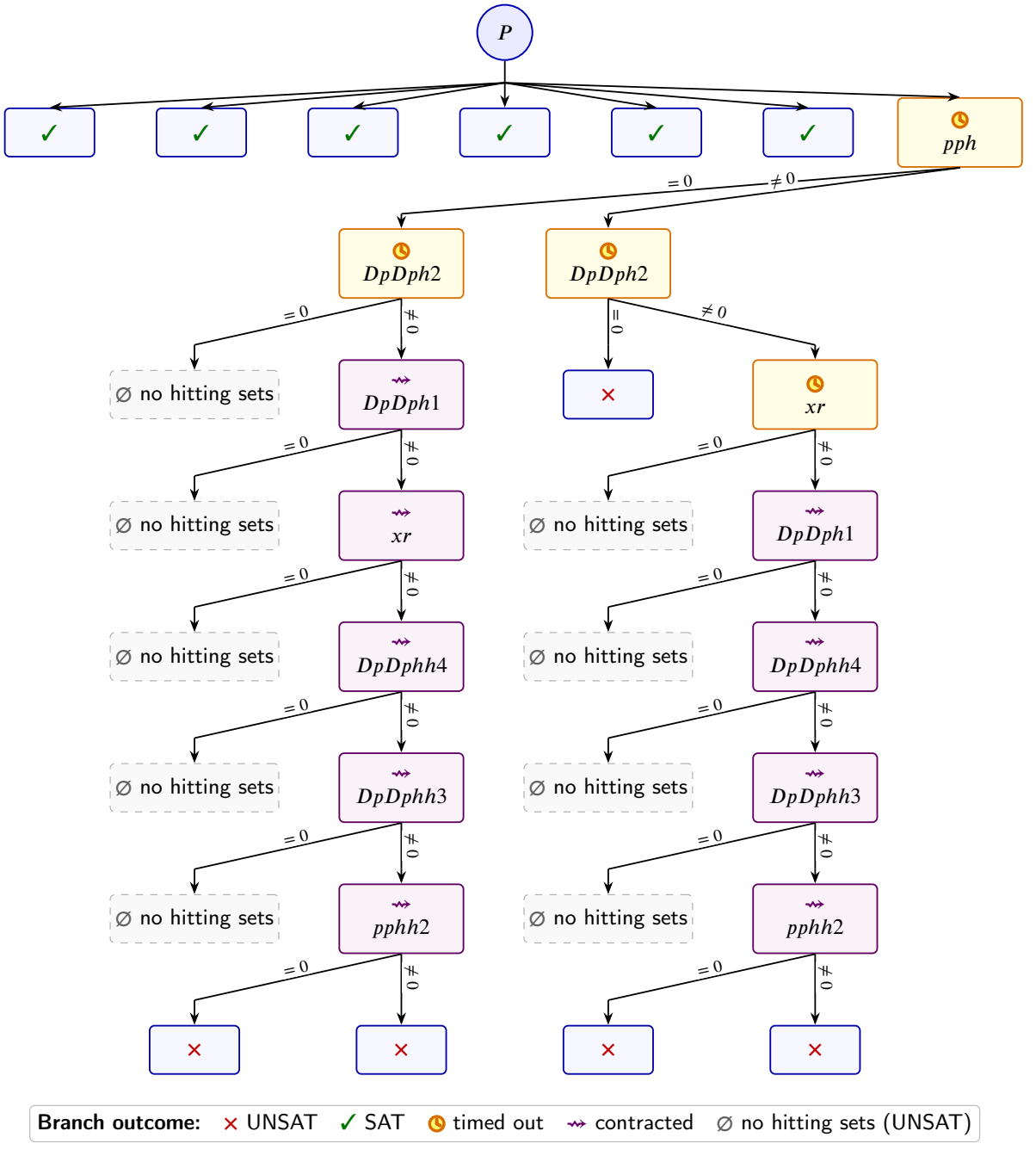


Figure 3: Split tree of factor branching with variable zeroing on the fifth-order graviton QFT system (detailed in Section 5.4) using M4GB with a five-second timeout per branch. The run takes 24.639 seconds. "contracted" branches are caused by an optimization specified in Subsection 4.3

4.3. Branch contraction

Suppose that one child of a split on x has no terminal branches (i.e., UNSAT) and the other has exactly one, H. Lemma 2 and Equation (3) give

$$V_Y(Q; Z, N) = \begin{cases} V_Y(H; Z, N \cup \{x\}) & \text{if the } x = 0 \text{ child is UNSAT,} \\ V_Y(H; Z \cup \{x\}, N) & \text{if the } x \neq 0 \text{ child is UNSAT.} \end{cases}$$

Thus we can choose the next splitting variable without running the branch solver on H, or losing solutions. This optimization may save a solver call that is likely to fail.

4.4. Theoretical analysis

The correctness argument uses only that the side conditions cover all possibilities. Let $Y = \mathrm{vars}(P)$ be the original variables, let A be the conditions on the current branch, and let $V_Y(Q; A)$ be the projection onto Y of the solutions of Q satisfying A. For variable zeroing, a pending branch (Q, Z, N) has $Q \subseteq K[Y \setminus Z][s_y : y \in N]$, and we write

$$V_Y(Q; Z, N) = \left\{ a \in \overline{K}^Y \,:\, a_z = 0 \,(z \in Z),\ a_y \neq 0 \,(y \in N),\ (a|_{Y \setminus Z}, s) \in V(Q) \text{ for some } s \right\}.$$

At the root, $V_Y(P; \emptyset, \emptyset) = V(P)$.

Theorem 4 (Correctness of an exhaustive split). *Let $D_1, \ldots, D_k$ be side conditions such that every point satisfying A satisfies at least one D_i. If each Q_i satisfies*

$$V_Y(Q_i; A \cup \{D_i\}) = \{a \in V_Y(Q; A) : a \text{ satisfies } D_i\},$$

then

$$V_Y(Q; A) = \bigcup_{i=1}^{k} V_Y(Q_i; A \cup \{D_i\}).$$

PROOF. Each set on the right is contained in $V_Y(Q; A)$ by hypothesis. Conversely, a point $a \in V_Y(Q; A)$ satisfies A, hence some D_i, and so lies in the i-th set on the right. □

For variable zeroing, the conditions $x = 0$ and $x \neq 0$ cover all values of x. Lemma 4 is a set-theoretic form of the decomposition $V(I) = V(I + \langle f \rangle) \cup \overline{V(I) \setminus V(f)}$ (Cox et al., 2015, Ch. 4, §4).

Lemma 4. *Let H be a terminal branch of a pending branch (Q, Z, N), let $Q_0 = H|_{x=0}$, and let $Q_1 =$* UpdateSaturation$(H, N \cup \{x\})$. *Then*

$$V_Y(H; Z, N) = V_Y(Q_0; Z \cup \{x\}, N) \cup V_Y(Q_1; Z, N \cup \{x\}). \tag{3}$$

PROOF. By Theorem 4, it suffices to show that Q_0 and Q_1 satisfy its hypothesis. For Q_0, a generator q satisfies $q|_{x=0}(b, s) = q(b, s)$ whenever $b_x = 0$, so a point lies in $V_Y(Q_0; Z \cup \{x\}, N)$ exactly when it lies in $V_Y(H; Z, N)$ and has $a_x = 0$. For Q_1, let H' be the auxiliary-free generators of H. Lemma 3 and Subsection 2.4 give $\pi(V(H)) = \{b \in V(H') : b_y \neq 0,\ y \in N\}$ and $\pi(V(Q_1)) = \{b \in V(H') : b_y \neq 0,\ y \in N \cup \{x\}\}$, so a point lies in $V_Y(Q_1; Z, N \cup \{x\})$ exactly when it lies in $V_Y(H; Z, N)$ and has $a_x \neq 0$. □

Lemma 2, applied in the ring of Q, gives $V(Q) = \bigcup_{H \in \mathcal{L}(Q)} V(H)$. Since $V_Y(Q; Z, N)$ depends on Q only through $V(Q)$,

$$V_Y(Q; Z, N) = \bigcup_{H \in \mathcal{L}(Q)} V_Y(H; Z, N). \tag{4}$$

Theorem 5 (Correctness of recursive splitting). *Suppose every split uses side conditions that cover all possibilities. If Algorithm 4 returns (S, U) on input P, then*

$$V(P) = \mathrm{Sol}(S) \cup \bigcup_{(H, Z, N) \in U} V_Y(H; Z, N),$$

where a representation in S stored with (Z, N) for the branch H represents $V_Y(H; Z, N)$.

PROOF. We show that the loop of Algorithm 4 preserves the invariant

$$V(P) = \mathrm{Sol}(S) \cup \bigcup_{(H,Z,N)\in U} V_Y(H; Z, N) \cup \bigcup_{(Q,Z,N)\in W} V_Y(Q; Z, N).$$

Before the loop, $S = U = \emptyset$ and $W = \{(P, \emptyset, \emptyset)\}$, so the invariant reads $V(P) = V_Y(P; \emptyset, \emptyset)$.

One iteration removes (Q, Z, N) from W. By Equation (4), $V_Y(Q; Z, N)$ is the union of $V_Y(H; Z, N)$ over $H \in \mathcal{L}(Q)$, and the iteration places each such set elsewhere. If $\mathcal{B}$ returns a representation of $V(H)$, the algorithm adds it with (Z, N) to S, so it represents $V_Y(H; Z, N)$. If $\mathcal{B}$ certifies H as UNSAT, then $V_Y(H; Z, N) = \emptyset$. If H is unresolved and not split, (H, Z, N) enters U. Otherwise the two children enter W, and their sets have union $V_Y(H; Z, N)$ by Equation (3). Hence the invariant still holds.

When the algorithm returns, $W = \emptyset$, and the invariant is the claimed equality. □

Corollary 2 (Completeness of variable zeroing). *If $U = \emptyset$ in the output of Algorithm 4, then* $\mathrm{Sol}(S) = V(P)$.

5. Experiments

5.1. Setup

We implemented Algorithm 3 in Rust 1.98.1 and used Symbolica 3.0.1 for polynomial arithmetic over the rationals (Ruijl, 2026). In all experiments, we solve the branches with a Gröbner-basis computation. The available implementations are M4GB, F5, GVW, F4, and Singular 4.4.1 (Decker et al., 2025; Faugère, 1999, 2002; Gao et al., 2016; Makarim and Stevens, 2017). A basis containing 1 certifies that the branch is UNSAT. The comparison also includes six direct whole-ideal Gröbner-basis configurations and three decomposition methods from Singular and Macaulay2 1.26.06 (Decker et al., 2025; Grayson and Stillman, 2026).

For Algorithm 4, we test M4GB, F5, GVW, F4, and Singular as branch solvers and impose a three-second limit on each hitting-set computation. We selected this cutoff by trying one, two, three, and five seconds on the hard gated systems. Three seconds is the smallest tested value that lets Singular complete the Katsura-8 residual (about 2.6 seconds); five seconds completes no additional hard system and only delays the subsequent splits.

We ran the benchmark on an Apple M4 Pro with 14 CPU cores and 24 GB of memory. Every table entry is a single run. Every method received 60 seconds per system; the three-second variable zeroing limit is an inner, per-branch limit within that budget.

For factor branching and variable zeroing, T is end-to-end wall time and F is the time to first SAT branch, measured from the start of the minimal-hitting-set search. Factorization and input processing are therefore included in T but not in F. The complete measurements are stored in the appendix.

5.2. Baseline systems

The baseline has 20 generated systems in six families.

- Signed-distance cycles and affine-factor hypergraphs test many inexpensive branches.
- The overlapping random multiples use seeded dense quadratic factors that recur across equations, with no factor shared by every equation.
- For gated systems, we create a new variable and multiply every equation in Cyclic, Katsura, or dense quadratic by it (Björck and Fröberg, 1991; Böge et al., 1986).
- This produces one easy selector branch and one nonlinear residual branch.

Table 2 selects the fastest completed factor branching backend and the fastest completed method in each baseline class. A factor branching entry gives T/F in seconds; the other entries give total wall time. "Timeout" means that every tested method in that class timed out.

Factor branching is a good choice when factorization produces branch ideals that are substantially easier than the expanded input ideal. A large number of hitting sets can still be manageable when individual branches are cheap to solve; the positive affine family shows that the hitting-set count alone is not decisive. The method is less effective when one branch retains the difficulty of the original nonlinear system, as Cyclic-7 does.

Family	Tested sizes	Variables/equations; hitting sets
Signed-distance cycle	$n = 12, 14, 16$	$12/12, 14/14, 16/16$; $924, 3432, 12870$
Overlapping random multiples	$(n, p, k) = (4, 6, 2), (6, 8, 3), (8, 10, 3)$	n/n; $4, 16, 28$
Positive affine hypergraph	$(n, m, k) = (8, 6, 4), (10, 8, 5), (12, 10, 6)$	n/m; $63, 239, 833$
Gated Cyclic	orders $5, 6, 7$	$6/5, 7/6, 8/7$; 2 each
Gated Katsura and dense quadratics	Katsura $6, 7, 8$; dense $n = 5, 6, 7$	$8/7, 9/8, 10/9, 6/5, 7/6, 8/7$; 2 each
Square affine hypergraph	$(n, k) = (8, 4), (10, 5)$	n/n; $178, 309$

Table 1
The generated systems used for the baseline benchmark.

System	Best factor branching T/F	Best direct GB	Best decomposition
Signed distance, $n = 12$	F5 2.669/0.006	**F4 0.443**	primdec 3.056
Signed distance, $n = 14$	GVW 9.824/0.006	**Singular 7.421**	primdec 29.894
Signed distance, $n = 16$	**M4GB 37.391/0.006**	timeout	timeout
Overlapping $(4, 6, 2)$	**F5 0.014/0.003**	F4 0.429	facstd 0.265
Overlapping $(6, 8, 3)$	**F5 0.160/0.006**	timeout	timeout
Overlapping $(8, 10, 3)$	**Singular 2.671/0.013**	timeout	timeout
Positive affine $(8, 6, 4)$	**M4GB 0.180/0.003**	timeout	facstd 0.368
Positive affine $(10, 8, 5)$	**F4 0.703/0.003**	timeout	facstd 1.836
Positive affine $(12, 10, 6)$	**F4 2.398/0.003**	timeout	facstd 53.318
Gated Cyclic-5	F4 0.011/0.003	**GVW 0.009**	facstd 0.181
Gated Cyclic-6	**Singular 0.043/0.010**	Singular 0.309	M2 2.620
Gated Cyclic-7	timeout	timeout	timeout
Gated Katsura-6	**Singular 0.047/0.010**	GVW 0.226	facstd 0.233
Gated Katsura-7	**Singular 0.283/0.010**	GVW 3.176	facstd 0.539
Gated Katsura-8	**Singular 2.575/0.010**	timeout	facstd 3.596
Gated dense, $n = 5$	**Singular 0.025/0.010**	M4GB 0.171	facstd 0.198
Gated dense, $n = 6$	**Singular 0.552/0.010**	M4GB 4.736	facstd 0.755
Gated dense, $n = 7$	**Singular 42.670/0.010**	timeout	facstd 48.820
Square affine $(8, 4)$	**M4GB 0.532/0.003**	timeout	facstd 0.601
Square affine $(10, 5)$	**F4 0.920/0.003**	timeout	facstd 2.141
#systems solved	19	9	16
#fastest (ties counted)	16	3	0

Table 2
Best completed baseline results across solver classes. All times are in seconds.

The measurements of time to first SAT branch support this explanation. On positive affine $(12, 10, 6)$, F is 0.003 seconds despite the 833 hitting sets. On gated Katsura-8, the first SAT branch is found in 0.010 seconds, but the full run takes 2.575 seconds. Factorization therefore finds the easy component quickly, while the nonlinear residual controls completion time.

5.3. Size-graded families

The baseline families were built to have factor structure. The following five families inherit it from the problem instead, and we grow each until every method times out.

- The reduced cyclic n-roots systems `Singular.rcyclic_n`, a variant of the cyclic n-roots problem (Björck and Fröberg, 1991), taken unchanged from SymbolicData[1].

[1] `https://github.com/symbolicdata/data`, commit `a5cdf53`.

- Nash equilibria of n-player games with two pure strategies per player (Sturmfels, 2002). Each player i either plays a pure strategy, $p_i(p_i - 1) = 0$, or is indifferent, so its payoff difference vanishes. The payoffs are random integers in $[-9, 9]$ (generic) or in $\{-1, 0, 1\}$ (degenerate).
- Critical points of a random quadratic program f on every face of the box $[0, 1]^n$, given by $x_i(x_i - 1)\,\partial_i f = 0$.
- Equilibria of a random Lotka–Volterra community on every species support (Hofbauer and Sigmund, 1998), given by $x_i\big(r_i + \sum_j a_{ij}x_j\big) = 0$ with $a_{ii} < 0$.

Each size is a single seeded instance. The setup is that of Subsection 5.1 with four differences. Factor branching entries give T only. The direct class adds Singular's `std` in degrevlex (Singular dp) and omits standalone F4, which remains a factor branching backend. A method that times out at one size is not run at larger sizes. Runs within a family are sequential, but some families ran concurrently, with at most five jobs on the 14 cores. We also compare with HomotopyContinuation.jl (HC.jl) (Breiding and Timme, 2018), which takes at least 26 seconds on every system, mostly for Julia start-up and compilation.

System	Best factor branching T	Best direct GB	Best decomposition	HC.jl
rcyclic-8	F4 0.019	**GVW 0.008**	facstd 0.425	—
rcyclic-10	**F4 0.018**	M4GB 0.041	timeout	—
rcyclic-12	**F4 0.021**	M4GB 0.198	timeout	—
rcyclic-14	**F4 0.029**	Singular dp 0.676	timeout	—
rcyclic-16	**Singular 0.062**	Singular dp 7.154	timeout	—
rcyclic-17	**Singular 0.091**	M2 19.76	timeout	—
rcyclic-18	**Singular 0.093**	M2 34.60	timeout	—
rcyclic-19	**Singular 0.113**	timeout	timeout	—
Nash generic, $n = 3$	F5 0.044	**F5 0.006**	facstd 0.197	29.45
Nash generic, $n = 4$	**GVW 0.136**	Singular dp 0.262	facstd 0.262	28.97
Nash generic, $n = 5$	**Singular 1.382**	timeout	timeout	34.79
Nash generic, $n = 6$	timeout	timeout	timeout	timeout
Nash degenerate, $n = 3$	M4GB 0.043	**GVW 0.005**	facstd 0.179	30.29
Nash degenerate, $n = 4$	**F5 0.086**	Singular lex 0.215	facstd 0.203	29.43
Nash degenerate, $n = 5$	**Singular 0.714**	timeout	timeout	34.84
Nash degenerate, $n = 6$	timeout	timeout	timeout	timeout
Box-QP, $n = 4$	F4 0.237	**F5 0.005**	facstd 0.179	26.66
Box-QP, $n = 5$	F5 0.700	**Singular dp 0.216**	facstd 0.297	31.49
Box-QP, $n = 6$	M4GB 2.174	**F5 0.033**	primdec 0.939	31.76
Box-QP, $n = 7$	**F5 6.494**	timeout	facstd 12.63	40.43
Box-QP, $n = 8$	**F5 20.42**	timeout	timeout	timeout
Box-QP, $n = 9$	timeout	timeout	timeout	timeout
Lotka–Volterra, $n = 6$	M4GB 0.191	**F5 0.006**	facstd 0.216	30.18
Lotka–Volterra, $n = 8$	F4 0.809	Singular dp 0.701	**facstd 0.464**	30.79
Lotka–Volterra, $n = 10$	**GVW 3.002**	timeout	facstd 6.432	35.40
Lotka–Volterra, $n = 12$	**F5 13.15**	timeout	timeout	54.45
Lotka–Volterra, $n = 14$	timeout	timeout	timeout	timeout
#systems solved	27	20	13	14
#fastest (ties counted)	18	8	1	0

Table 3
Best completed results across solver classes on the size-graded families, with a 60-second limit. For rcyclic, only selected sizes are shown; the summary rows count all 31 systems, rcyclic-8 to rcyclic-19 included. HC.jl was not run on the positive-dimensional rcyclic systems. All times are in seconds.

Table 3 shows the same pattern in every family. The baselines win only on small systems, each of which they finish in under 0.5 seconds. From rcyclic-10, $n = 4$ for the Nash games, $n = 7$ for box-QP, and $n = 10$ for Lotka–Volterra, factor branching is fastest, and its lead grows with n: on rcyclic it rises from 2.3 times at $n = 10$ to 372 times at $n = 18$. Only factor branching completes rcyclic-19 and box-QP with $n = 8$. The two families reach this point differently. Every

rcyclic system refines into only 3 terminal branches, each far easier than the whole system. Box-QP refines into up to 3^n branches, 6534 at $n = 8$, and each branch is cheap, as in the positive affine family.

Factor branching can also use a decomposition method as its branch solver. Table 4 compares each of the three decomposition methods as a backend with the same method alone.

	`facstd`		primdec		M2 `decompose`	
System	FB	alone	FB	alone	FB	alone
rcyclic-8	**0.038**	0.425	**0.264**	timeout	1.247	**1.120**
rcyclic-10	**0.040**	timeout	timeout	timeout	**1.569**	timeout
rcyclic-12	**0.043**	timeout	timeout	timeout	**1.999**	timeout
rcyclic-14	**0.053**	timeout	timeout	timeout	**3.326**	timeout
rcyclic-16	**0.079**	timeout	timeout	timeout	**6.341**	timeout
rcyclic-17	**0.119**	timeout	timeout	timeout	**11.92**	timeout
rcyclic-18	**0.162**	timeout	timeout	timeout	**15.60**	timeout
rcyclic-19	**0.285**	timeout	timeout	timeout	**33.82**	timeout
Nash generic, $n = 3$	**0.138**	0.197	**0.205**	0.206	4.746	**0.588**
Nash generic, $n = 4$	0.449	**0.262**	**0.665**	timeout	15.72	**3.560**
Nash generic, $n = 5$	**13.48**	timeout	timeout	timeout	timeout	timeout
Nash degenerate, $n = 3$	**0.136**	0.179	0.207	**0.202**	4.760	**0.525**
Nash degenerate, $n = 4$	0.263	**0.203**	**0.433**	4.766	10.21	**0.789**
Nash degenerate, $n = 5$	**1.634**	timeout	timeout	timeout	timeout	timeout
Box-QP, $n = 4$	0.661	**0.179**	1.085	**0.226**	29.03	**0.806**
Box-QP, $n = 5$	1.928	**0.297**	**3.484**	14.90	timeout	**55.87**
Box-QP, $n = 6$	7.528	**1.259**	11.46	**0.939**	timeout	**11.58**
Box-QP, $n = 7$	22.77	**12.63**	**35.12**	timeout	timeout	timeout
Lotka–Volterra, $n = 6$	0.615	**0.216**	0.930	**0.265**	20.70	**0.872**
Lotka–Volterra, $n = 8$	3.012	**0.464**	**4.826**	timeout	timeout	timeout
Lotka–Volterra, $n = 10$	10.05	**6.432**	**14.99**	timeout	timeout	timeout
Lotka–Volterra, $n = 12$	**41.47**	timeout	timeout	timeout	timeout	timeout

Table 4
Each decomposition method as the factor branching (FB) backend and alone, with a 60-second limit; the faster of each pair is in bold. A method that timed out at a smaller size of a family was not run again and is listed as a timeout. Sizes at which every method times out are omitted. All times are in seconds.

Factor branching with `facstd` completes every rcyclic system in under 0.3 seconds, whereas `facstd` alone times out from rcyclic-10. It also completes the largest Nash and Lotka–Volterra systems that factor branching solves, on which `facstd` alone times out. On box-QP, the $n = 4$ Nash games, and the smaller Lotka–Volterra systems, `facstd` alone is faster. There, factor branching starts Singular once per terminal branch, whereas `facstd` alone starts it once. With primdec, factor branching completes rcyclic-8, Nash generic with $n = 4$, box-QP with $n = 7$, and Lotka–Volterra with $n = 8$ and $n = 10$, on which primdec alone times out; it times out itself from rcyclic-10 and at $n = 5$ for the Nash games. The Macaulay2 backend likewise starts Macaulay2 once per branch. It beats `decompose` alone from rcyclic-10 to rcyclic-19, where there are only 3 branches, but loses on the application families, for example 20.70 against 0.872 seconds on the 58 branches of Lotka–Volterra with $n = 6$.

5.4. Physical System

The system is an order-five approximation constraining graviton couplings in asymptotic-safety models to remain consistent with quantum field theory (QFT) (Rafie-Zinedine, 2018; Vacca and Zanusso, 2010). Solving this system helps determine admissible fixed-point structures in quantum gravity (Vacca and Zanusso, 2010).

For this larger system, every run, including the standalone baselines, has a 120-second total limit. Within it, variable zeroing uses a five-second limit per branch; factor branching places no further limit on a branch.

Backend	Factor branching T/F (s)	Factor branching result	Variable zeroing T/F (s)	Variable zeroing result
M4GB	*timeout*/0.003	partial: 6 SAT, 1 timeout	24.710/0.432	complete: 13 splits
F5	*timeout*/0.006	partial: 6 SAT, 1 timeout	*timeout*/0.421	partial: 35 splits, 11 unresolved
GVW	*timeout*/0.006	partial: 6 SAT, 1 timeout	*timeout*/0.490	partial: 35 splits, 11 unresolved
F4	*timeout*/0.006	partial: 6 SAT, 1 timeout	*timeout*/0.434	partial: 32 splits, 8 unresolved
Singular	*timeout*/0.013	partial: 6 SAT, 1 timeout	17.791/0.434	complete: 7 splits
`facstd`	0.785/0.010	complete: 6 SAT, 1 UNSAT	1.207/0.425	complete: 0 splits

Table 5
Physical-system results with a 120-second total limit per run; variable zeroing also uses a 5-second limit per branch. Both methods refine the system into the same 7 terminal branches; the factor branching result column counts them by outcome. A split is one $x = 0$ versus $x \neq 0$ case split; an unresolved branch is one still open when the total limit expires. The nine standalone baselines, including standalone `facstd`, all time out and are omitted.

All nine standalone baselines time out after 120 seconds on the physical system: M4GB, F5, GVW, and F4 in grevlex order, Singular and Macaulay2 lex Gröbner bases, Singular `facstd` and `primdecGTZ`, and Macaulay2 `decompose`. Factor branching with `facstd` completes in 0.785 seconds. Every backend solves the same 6 of the 7 terminal branches as SAT within 0.07 seconds in total, and times out on the last one. Only `facstd` shows that this branch is UNSAT, so only in hindsight do we know that the 6 SAT branches found by the other backends already contain every solution.

Variable zeroing also completes with M4GB and Singular, making completion more robust to backend choice on this system: three of six backends succeed, versus one with factor branching alone.

6. Conclusion

We introduced a solver-agnostic framework that decomposes polynomial systems into smaller branches by enumerating minimal hitting sets of their factor sets. The framework can recursively split timed-out branches into the cases $x = 0$ and $x \neq 0$.

We proved that refinement terminates and preserves the full solution set. Variable zeroing also preserves the solution set and terminates provided that each timed solver call returns. The output is complete whenever no branches remain unresolved.

On the baseline benchmarks, factor branching completed 19 of 20 systems, compared with 9 for direct Gröbner-basis methods and 16 for decomposition methods. On the QFT system, variable zeroing extends certified completion from `facstd` alone to M4GB and Singular, improving robustness to backend choice: three of six backends complete, versus one with factor branching alone.

Future work will test this benefit on other systems, and improve the variable choice for splitting or find better splitting side conditions.

CRediT authorship contribution statement

Ilay Menahem: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Michael Shalyt:** Writing – review & editing. **Ido Kaminer:** Writing – review & editing, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code and data associated with this work are publicly available at `https://github.com/IlayMenahem/soothing`.

Acknowledgments

This work did not receive any dedicated funding, and the authors have no one to acknowledge beyond those credited as authors above.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used generative AI tools to assist with drafting and editing portions of the manuscript text. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Berge, C., 1989. Hypergraphs: Combinatorics of Finite Sets. North-Holland, Amsterdam.

Björck, G., Fröberg, R., 1991. A faster way to count the solutions of inhomogeneous systems of algebraic equations, with applications to cyclic *n*-roots. Journal of Symbolic Computation 12, 329–336. doi:10.1016/S0747-7171(08)80153-8.

Böge, W., Gebauer, R., Kredel, H., 1986. Some examples for solving systems of algebraic equations by calculating Gröbner bases. Journal of Symbolic Computation 2, 83–98. doi:10.1016/S0747-7171(86)80014-1.

Breiding, P., Timme, S., 2018. HomotopyContinuation.jl: A package for homotopy continuation in Julia, in: Mathematical Software – ICMS 2018, Springer. pp. 458–465. doi:10.1007/978-3-319-96418-8_54.

Buchberger, B., 2006. Bruno Buchberger's PhD thesis 1965: An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal. Journal of Symbolic Computation 41, 475–511. doi:10.1016/j.jsc.2005.09.007.

Cox, D.A., Little, J., O'Shea, D., 2015. Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra. 4 ed., Springer, Cham. doi:10.1007/978-3-319-16721-3.

Decker, W., Greuel, G.M., Pfister, G., Schönemann, H., 2025. Singular 4-4-1: A computer algebra system for polynomial computations. https://www.singular.uni-kl.de.

Eiter, T., Gottlob, G., 1995. Identifying the minimal transversals of a hypergraph and related problems. SIAM Journal on Computing 24, 1278–1304. doi:10.1137/S0097539793250299.

Faugère, J.C., 1999. A new efficient algorithm for computing Gröbner bases (F4). Journal of Pure and Applied Algebra 139, 61–88. doi:10.1016/S0022-4049(99)00005-5.

Faugère, J.C., 2002. A new efficient algorithm for computing Gröbner bases without reduction to zero (F5), in: Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation, ACM. pp. 75–83. doi:10.1145/780506.780516.

Gao, S., Volny, IV, F., Wang, M., 2016. A new framework for computing Gröbner bases. Mathematics of Computation 85, 449–465. doi:10.1090/MCOM/2969.

Gianni, P., Trager, B., Zacharias, G., 1988. Gröbner bases and primary decomposition of polynomial ideals. Journal of Symbolic Computation 6, 149–167. doi:10.1016/S0747-7171(88)80040-3.

Gräbe, H.G., 1995. On factorized Gröbner bases, in: Fleischer, J., Grabmeier, J., Hehl, F.W., Küchlin, W. (Eds.), Computer Algebra in Science and Engineering. World Scientific, Singapore, pp. 77–89. URL: https://www.informatik.uni-leipzig.de/~graebe/ComputerAlgebra/Publications/fgb.pdf.

Grayson, D.R., Stillman, M.E., 2026. Macaulay2, a software system for research in algebraic geometry. https://macaulay2.com. Version 1.26.06.

Hofbauer, J., Sigmund, K., 1998. Evolutionary Games and Population Dynamics. Cambridge University Press, Cambridge.

Kera, H., Ishihara, Y., Kambe, Y., Vaccon, T., Yokoyama, K., 2024. Learning to compute Gröbner bases, in: Advances in Neural Information Processing Systems. URL: https://openreview.net/forum?id=ZRz7XlxBzQ.

Kőnig, D., 1927. Über eine Schlussweise aus dem Endlichen ins Unendliche. Acta Scientiarum Mathematicarum (Szeged) 3, 121–130.

Kreuzer, M., Robbiano, L., 2000. Computational Commutative Algebra 1. Springer, Berlin. doi:10.1007/978-3-540-70628-1.

Makarim, R.H., Stevens, M., 2017. M4GB: An efficient Gröbner-basis algorithm, in: Proceedings of the 2017 ACM International Symposium on Symbolic and Algebraic Computation, ACM. pp. 293–300. doi:10.1145/3087604.3087638.

Mayr, E.W., Meyer, A.R., 1982. The complexity of the word problems for commutative semigroups and polynomial ideals. Advances in Mathematics 46, 305–329. doi:10.1016/0001-8708(82)90048-2.

Peifer, D., Stillman, M., Halpern-Leistner, D., 2020. Learning selection strategies in Buchberger's algorithm, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 7575–7585. URL: https://proceedings.mlr.press/v119/peifer20a.html.

Rabinowitsch, J.L., 1930. Zum Hilbertschen Nullstellensatz. Mathematische Annalen 102, 520. doi:10.1007/BF01782361.

Rafie-Zinedine, S., 2018. Simplifying Quantum Gravity Calculations. Master's thesis. Lund University. ArXiv:1808.06086.

Ruijl, B., 2026. Symbolica: Modern computer algebra. https://symbolica.io. Version 3.

Sommese, A.J., Verschelde, J., Wampler, C.W., 2001. Numerical decomposition of the solution sets of polynomial systems into irreducible components. SIAM Journal on Numerical Analysis 38, 2022–2046. doi:10.1137/S0036142900372549.

Sommese, A.J., Wampler, II, C.W., 2005. The Numerical Solution of Systems of Polynomials Arising in Engineering and Science. World Scientific, Singapore. doi:10.1142/5763.

Sturmfels, B., 2002. Solving Systems of Polynomial Equations. volume 97 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI. doi:10.1090/cbms/097.

Vacca, G.P., Zanusso, O., 2010. Asymptotic safety in einstein gravity and scalar-fermion matter. Physical Review Letters 105, 231601. doi:10.1103/PhysRevLett.105.231601.

A. Complete experimental results

The tables below give every run summarized in Table 2. Unless stated otherwise, entries are end-to-end wall times in seconds. P marks a partial run, which reached the 60-second limit after finding at least one SAT branch, and TO a run that reached the 60-second limit before reporting any result. These are single measurements.

A.1. Factor branching

Table 6 reports all five branch-solver configurations. A partial entry found the easy component shown in the main text but left at least one branch unresolved.

Table 6
End-to-end wall times for every factor-branching configuration.

System	M4GB	F5	GVW	F4	Singular
Signed distance, $n = 12$	2.847	2.669	2.716	2.670	8.460
Signed distance, $n = 14$	9.914	9.894	9.824	9.887	30.975
Signed distance, $n = 16$	37.391	46.356	TO	TO	TO
Overlapping $(4, 6, 2)$	0.024	0.014	0.019	0.024	0.041
Overlapping $(6, 8, 3)$	1.032	0.160	0.355	0.687	0.181
Overlapping $(8, 10, 3)$	TO	10.732	35.113	TO	2.671
Positive affine $(8, 6, 4)$	0.180	0.189	0.185	0.209	0.571
Positive affine $(10, 8, 5)$	0.717	0.707	0.711	0.703	2.063
Positive affine $(12, 10, 6)$	2.404	2.401	2.410	2.398	7.884
Gated Cyclic-5	0.011	0.011	0.011	0.011	0.020
Gated Cyclic-6	P 60.007	5.653	2.198	P 60.007	0.043
Gated Cyclic-7	P 60.005	P 60.364	P 61.200	P 60.009	P 60.025
Gated Katsura-6	0.348	0.502	1.377	2.628	0.047
Gated Katsura-7	4.247	15.338	42.858	P 60.007	0.283
Gated Katsura-8	51.074	P 60.085	P 60.021	P 60.005	2.575
Gated dense, $n = 5$	0.174	0.074	0.203	0.209	0.025
Gated dense, $n = 6$	4.708	3.216	7.477	21.756	0.552
Gated dense, $n = 7$	P 60.008	P 60.046	P 60.032	P 60.004	42.670
Square affine $(8, 4)$	0.532	0.557	0.540	0.557	1.569
Square affine $(10, 5)$	0.940	0.975	0.927	0.920	2.909

A.2. Variable zeroing

Table 7 gives, for each system, the fastest completed variable zeroing configuration; where no configuration completes, it gives the partial run with the fewest unresolved branches. Every backend receives three seconds per branch and a 60-second outer budget. Here N is the number of recursion nodes, E the number of hitting-set evaluations, O/S the numbers of per-branch timeouts and variable zeroing splits, and C/U the numbers of returned components and unresolved branches. The time entry is T/F.

On the three signed-distance systems and positive affine $(12, 10, 6)$, refinement of the root system exceeds the three-second per-branch limit, so no hitting set is evaluated. With a 60-second per-branch limit this refinement finishes; for example, signed distance $n = 12$ refines to 924 hitting sets in 23.4 seconds and positive affine $(12, 10, 6)$ to 833 in 4.8 seconds.

Table 7
For each system, the fastest completed variable-zeroing configuration or, if none completes, the partial run with the fewest unresolved branches. Times are end-to-end wall time and time to first SAT branch in seconds.

System	Backend	T/F	N	E	O/S	C/U	Result
Signed distance, $n = 12$	F5	3.000/–	1	0	0/0	0/1	partial
Signed distance, $n = 14$	GVW	3.000/–	1	0	0/0	0/1	partial
Signed distance, $n = 16$	GVW	3.000/–	1	0	0/0	0/1	partial
Overlapping $(4, 6, 2)$	F5	0.051/0.037	1	4	0/0	4/0	complete
Overlapping $(6, 8, 3)$	F5	0.274/0.127	1	16	0/0	16/0	complete
Overlapping $(8, 10, 3)$	Singular	2.931/0.258	1	28	0/0	28/0	complete
Positive affine $(8, 6, 4)$	M4GB	0.642/0.435	1	63	0/0	63/0	complete
Positive affine $(10, 8, 5)$	M4GB	2.408/1.608	1	239	0/0	239/0	complete
Positive affine $(12, 10, 6)$	M4GB	3.001/–	1	0	0/0	0/1	partial
Gated Cyclic-5	M4GB	0.030/0.021	1	2	0/0	2/0	complete
Gated Cyclic-6	Singular	0.059/0.026	1	2	0/0	2/0	complete
Gated Cyclic-7	Singular	15.531/0.026	15	10	5/7	1/1	partial
Gated Katsura-6	Singular	0.063/0.027	1	2	0/0	2/0	complete
Gated Katsura-7	Singular	0.295/0.025	1	2	0/0	2/0	complete
Gated Katsura-8	Singular	2.599/0.027	1	2	0/0	2/0	complete
Gated dense, $n = 5$	Singular	0.049/0.033	1	2	0/0	2/0	complete
Gated dense, $n = 6$	Singular	0.552/0.023	1	2	0/0	2/0	complete
Gated dense, $n = 7$	F4	60.041/0.020	12	8	7/6	1/7	partial
Square affine $(8, 4)$	M4GB	1.532/1.029	1	178	0/0	178/0	complete
Square affine $(10, 5)$	F4	2.659/1.778	1	309	0/0	309/0	complete

A.3. Direct whole-ideal Gröbner bases

Table 8 reports the six direct whole-ideal Gröbner-basis configurations described in Subsection 5.1.

Table 8
End-to-end wall times for every direct whole-ideal Gröbner-basis configuration.

System	M4GB	F5	GVW	F4	Singular	Macaulay2
Signed distance, $n = 12$	3.266	20.044	8.685	0.472	0.512	1.293
Signed distance, $n = 14$	TO	TO	TO	TO	7.421	33.164
Signed distance, $n = 16$	TO	TO	TO	TO	TO	TO
Overlapping $(4, 6, 2)$	1.000	24.865	0.472	0.413	TO	TO
Overlapping $(6, 8, 3)$	TO	TO	TO	TO	TO	TO
Overlapping $(8, 10, 3)$	TO	TO	TO	TO	TO	TO
Positive affine $(8, 6, 4)$	TO	TO	TO	TO	TO	TO
Positive affine $(10, 8, 5)$	TO	TO	TO	TO	TO	TO
Positive affine $(12, 10, 6)$	TO	TO	TO	TO	TO	TO
Gated Cyclic-5	0.012	0.279	0.009	0.008	0.183	0.480
Gated Cyclic-6	TO	TO	1.738	TO	0.309	0.917
Gated Cyclic-7	TO	TO	TO	TO	TO	TO
Gated Katsura-6	0.436	44.961	0.226	5.277	14.430	TO
Gated Katsura-7	5.165	TO	3.176	TO	TO	TO
Gated Katsura-8	TO	TO	TO	TO	TO	TO
Gated dense, $n = 5$	0.171	8.884	0.179	0.200	0.529	TO
Gated dense, $n = 6$	4.736	TO	6.755	17.472	TO	TO
Gated dense, $n = 7$	TO	TO	TO	TO	TO	TO
Square affine $(8, 4)$	TO	TO	TO	TO	TO	TO
Square affine $(10, 5)$	TO	TO	TO	TO	TO	TO

A.4. Decomposition methods

Table 9 reports Singular's `facstd` and `primdecGTZ`, and Macaulay2's `decompose`.

Table 9
End-to-end wall times for every decomposition configuration.

System	facstd	primdecGTZ	Macaulay2
Signed distance, $n = 12$	4.800	3.056	25.223
Signed distance, $n = 14$	TO	29.894	TO
Signed distance, $n = 16$	TO	TO	TO
Overlapping $(4, 6, 2)$	0.265	TO	TO
Overlapping $(6, 8, 3)$	TO	TO	TO
Overlapping $(8, 10, 3)$	TO	TO	TO
Positive affine $(8, 6, 4)$	0.368	TO	TO
Positive affine $(10, 8, 5)$	1.836	TO	TO
Positive affine $(12, 10, 6)$	53.318	TO	TO
Gated Cyclic-5	0.181	0.337	0.709
Gated Cyclic-6	2.729	TO	2.620
Gated Cyclic-7	TO	TO	TO
Gated Katsura-6	0.233	TO	TO
Gated Katsura-7	0.539	TO	TO
Gated Katsura-8	3.596	TO	TO
Gated dense, $n = 5$	0.198	TO	TO
Gated dense, $n = 6$	0.755	TO	TO
Gated dense, $n = 7$	48.820	TO	TO
Square affine $(8, 4)$	0.601	TO	TO
Square affine $(10, 5)$	2.141	TO	TO